Nelson Open - Congruence Closure DAG - SAT Solver

Silver Gjeka

January 30, 2025

University: University of Verona (UNIVR)
Subject: Automated Reasoning
Degree: Master's Degree in Artificial Intelligence
Matriculated: VR527645

Contents

1 Introduction

This project focuses on developing a solver that determines whether a set of logical formulas can be satisfied. The formulas belong to three logical theories: equality (with free symbols), non-empty possibly cyclic lists, and arrays (without extensionality). The core of the solver is a congruence closure algorithm applied to Directed Acyclic Graphs (DAGs). This algorithm checks if the given equalities and inequalities can be consistently satisfied.

To improve the performance and efficiency of the solver, we have incorporated several heuristics:

- **Forbidden List or Set**: This heuristic helps avoid problematic combinations of elements that may lead to contradictions or logical errors.
- **Choosing the Representative in the UNION Function**: When merging groups of elements, the solver selects the element with the largest set of related elements. This helps ensure better decision-making when performing merges.
- **Non-Recursive FIND Function**: This version of the FIND function eliminates recursion, which increases efficiency by ensuring that all terms in a group are updated correctly when merging.

These improvements may not always help in every situation, but they are meant to make the solver faster and more accurate. Since their effects can vary, you can choose to test them and see how they impact the solver's performance.

1.1 Formula Transformations

To handle a variety of formulas, the solver applies several transformations to simplify them before processing. If any unsupported symbols are found, an error will be raised, indicating that those symbols are not accepted.

- Free predicate symbols are adjusted where necessary to fit the solver's needs.
- Non-conjunctive formulas are converted to Disjunctive Normal Form (DNF), making them easier to process.
- Arithmetic symbols and other unsupported symbols are replaced with free symbols. If the formula contains symbols that are not supported, an error will be triggered, notifying that these symbols are not accepted.
- Quantifiers are removed, and variables are treated as free symbols, allowing the solver to handle them more easily.

These transformations ensure that the formulas are in a standardized format, making it easier for the solver to determine their satisfiability.

2 Syntax

- **Variables:**
  - General variables: Represented by symbols like `x`, `y`, `z`. These are commonly used as arguments, e.g., `Fn(x)` or `car(x)`.
  - Functions: Denoted by `Fn(...)` or `Hn(...)` for operations within formulas.
  - Atoms and Predicates:
    - `atom(...)`: Represents a single atomic formula.
    - `LEAVE(...)`: A predicate representing an action or state, e.g., "a is leaving."
  - List theory symbols:
    - `car(.)`: Represents the head (first element) of a list.
    - `cdr(.)`: Represents the tail (remaining elements) of a list.
    - `cons(arg1, arg2)`: Creates a list from a head (`arg1`) and a tail (`arg2`).
  - Arrays:
    - `select(arg1, arg2)`: Retrieves a value from an array (`arg1` is the array, `arg2` the index).
    - `store(arg1, arg2, arg3)`: Stores a value (`arg3`) in an array at index `arg2`. Creates the array if it does not exist.
- **Arguments:**
  - Functions: Can take multiple arguments, e.g., `Fn(x, y, z)`.
  - Lists:
    - `car(arg)`: Retrieves the head of a valid list.
    - `cdr(arg)`: Retrieves the tail of a valid list.
  - Arrays:
    - `select(arg1, arg2)`: `arg1` is the array name, `arg2` the index.
    - `store(arg1, arg2, arg3)`: `arg1` is the array name, `arg2` the index, and `arg3` the value.

2.1 DNF (Disjunctive Normal Form):

- **Logical Operators, Equality, Inequality:**
  - `&`: Conjunction (AND),
  - `|`: Disjunction (OR),
  - `=`: Equality,
  - `!=`: Inequality,
  - `->`: Implication,
  - $\sim$: Negation.

- **Restrictions:**
  - The formula must be written in Negation Normal Form (NNF) before transformation.
  - The biconditional (`<->`) should be expressed as `(a -> b) & (b -> a)`.
  - Pay attention to the use of parentheses in the formula:
    * For equality formulas without negations, you may omit parentheses, e.g., `a = b` or `a != b`.
    * If negations are involved, they must be formatted as follows: for a single variable, use (∼`(a)`), and for equality/inequality, use ∼`(a != b)`.
    * When specifying transformation actions, use parentheses to indicate the sequence of operations, e.g., `(a | (a & b))`, where the `&` operation should be performed before the `|` operation.
  - Make sure that you devide by a space the = and != assigmet. `Example:` (a = b) and not (a=b).
  - Be sure to follow these formatting guidelines precisely. Incorrect formatting may result in errors during the transformation process. For reference, you can check the `inputFormulas` folder for examples.

2.2 DAG (Directed Acyclic Graph):

- **Negation:** Use `-(...)` for negation, e.g., `-a`, `-a != b`.
- **Conjunction:** Use `;` instead of `&` for conjunction.
- **Restrictions:**
  - A function, once defined with one argument (e.g., `Fn(x)`), cannot be redefined with more arguments (e.g., `Fn(x, y)`).
  - Unsupported symbols, such as arithmetic symbols, cannot be used in the formula. If these symbols are included, the program will trigger an error during compilation, indicating that the symbols are not accepted.

3 Project Functionality

The project is based on the Nelson-Oppen method, which handles all three theories in the formula, determining its satisfiability.

Warning

The input formulas must be written in *Negation Normal Form (NNF)* before being processed by the project. For detailed guidelines on how to properly format formulas, refer to Section 2: Syntax. Incorrectly formatted formulas may result in errors or invalid results.

- *Disjunctive Normal Form (DNF) Transformation:* This option allows the input formula to be transformed into Disjunctive Normal Form (DNF), which simplifies the structure of the formula for easier analysis.

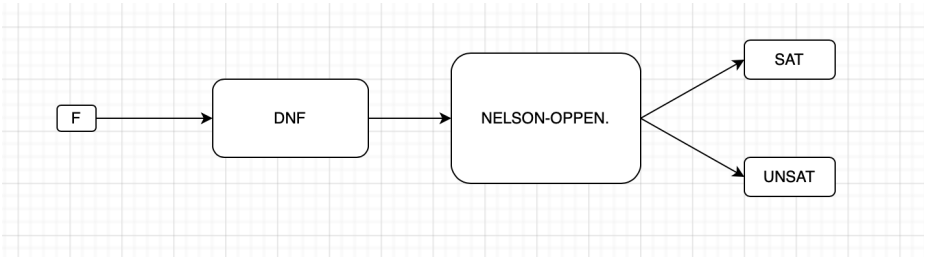

Figure 1: Project Schema

- *DAG-Based Congruence Closure Algorithm:* Using the Nelson-Oppen method, the project integrates three key logical theories: **Equality**, **Arrays**, and **Lists** to reason about the input formula. The algorithm constructs a Directed Acyclic Graph (DAG), applies congruence closure, and checks for satisfiability. This process ensures the formula is logically consistent across all supported theories.

3.1 DNF:

Step 1: The formula is parsed, and if any quantifiers are present, they are removed. This means that universal quantifiers ($\forall$) and existential quantifiers ($\exists$) are eliminated from the formula, simplifying the structure.

Consider the following formula with quantifiers:

$$\forall x \, \exists y \, (Pn(x, y) \wedge Qn(x, y))$$

In this formula, the quantifiers $\forall x$ and $\exists y$ are present. In Step 1, we remove the quantifiers, which results in the simplified formula:

$$P(x, y) \wedge Q(x, y)$$

Step 2 & 3: The equalities and disequalities in the formula are mapped to simplify its structure. For instance, consider the formula:

$$\text{car}(x) \neq \text{select}(\text{store}(a, i, v), i)$$

This is mapped as:

$$f_0 \rightarrow \text{car}(x)$$
$$f_1 \rightarrow \text{select}(\text{store}(a, i, v), i)$$

Hence, the formula becomes:

$$f_0 \neq f_1$$

Next, this mapping is further simplified:

$$e_0 \rightarrow f_0 \neq f_1$$

The final result of this mapping is e_0.

Step 4: The formula $(a \vee b)$ is in Conjunctive Normal Form (CNF). To convert it to Disjunctive Normal Form (DNF), we need to create a truth table for this formula and then use the truth values to identify the DNF expression.

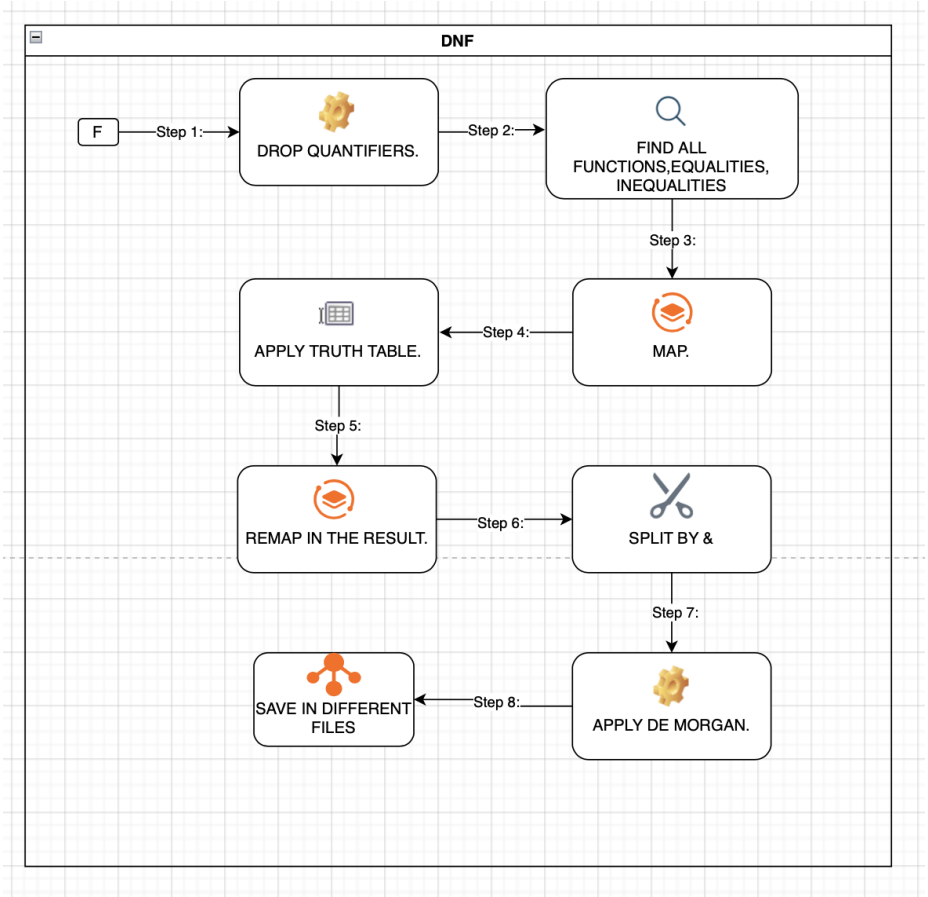

Figure 2: DNF Transformation Process

a	**b**	**a ∨ b**
True	True	True
True	False	True
False	True	True
False	False	False

From the truth table, we identify the rows where $(a \vee b)$ is True (i.e., the first three rows). For each of these rows, we create a conjunction of literals (variables or negated variables):

- Row 1: a = True, b = True → a ∧ b
- Row 2: a = True, b = False → a∧ ∼ b
- Row 3: a = False, b = True → ∼ a ∧ b

The DNF expression is the disjunction (OR) of these three terms:

$$\text{DNF} = (\text{a} \wedge \text{b}) \vee (\text{a} \wedge \sim \text{b}) \vee (\sim \text{a} \wedge \text{b})$$

Step 5: Remapping Results to DNF.
Let's consider the simplified formula e_0, which was previously mapped in earlier steps.
This formula gets mapped to a new equivalence e_0, which we can write as:

$$e_0 \equiv f_0 \neq f_1$$

The mapping of f_0, f_1 is:

$$f_0 \rightarrow \text{car}(x)$$

$$f_1 \rightarrow \text{select}(\text{store}(a, i, v), i)$$

Thus, the final result for this formula is:

$$e_0 \equiv \text{car}(x) \neq \text{select}(\text{store}(a, i, v), i)$$

Step 6: Splitting the formulas by | ;.
If a formula contains a disjunction like $a|b$, it will be split into two separate formulas:

$$\text{Formula 1: } a$$

$$\text{Formula 2: } b$$

Step 7: Applying De Morgan's Law when encountering negations of equalities.
If the formula contains a negation of an equality, such as $\sim (a = b)$, it will be transformed into an inequality:

$$\sim (a = b) \rightarrow a \neq b$$

Step 8: Saving the spliced formulas in different files.
Each split formula from previous steps will be saved in its own separate file for easier handling. For example:

$$\text{File 1: } a$$

$$\text{File 2: } b$$

For more information on the DNF transformation, check the link: Disjunctive Normal Form on Wikipedia.

3.2 NELSON-OPPEN & Congruence Closure Algorithm DAG

The Nelson-Oppen and Congruence Closure Algorithm works with formulas stored in the `alreadyToDnf/` folder. This folder contains formulas that have been converted to Disjunctive Normal Form (DNF) as well as test files added manually. The algorithm reads one of these files to check if the formulas can be satisfied. This process makes sure the transformation and checking of the formulas are done in an organized way.

- The algorithm is executed in several structured steps:

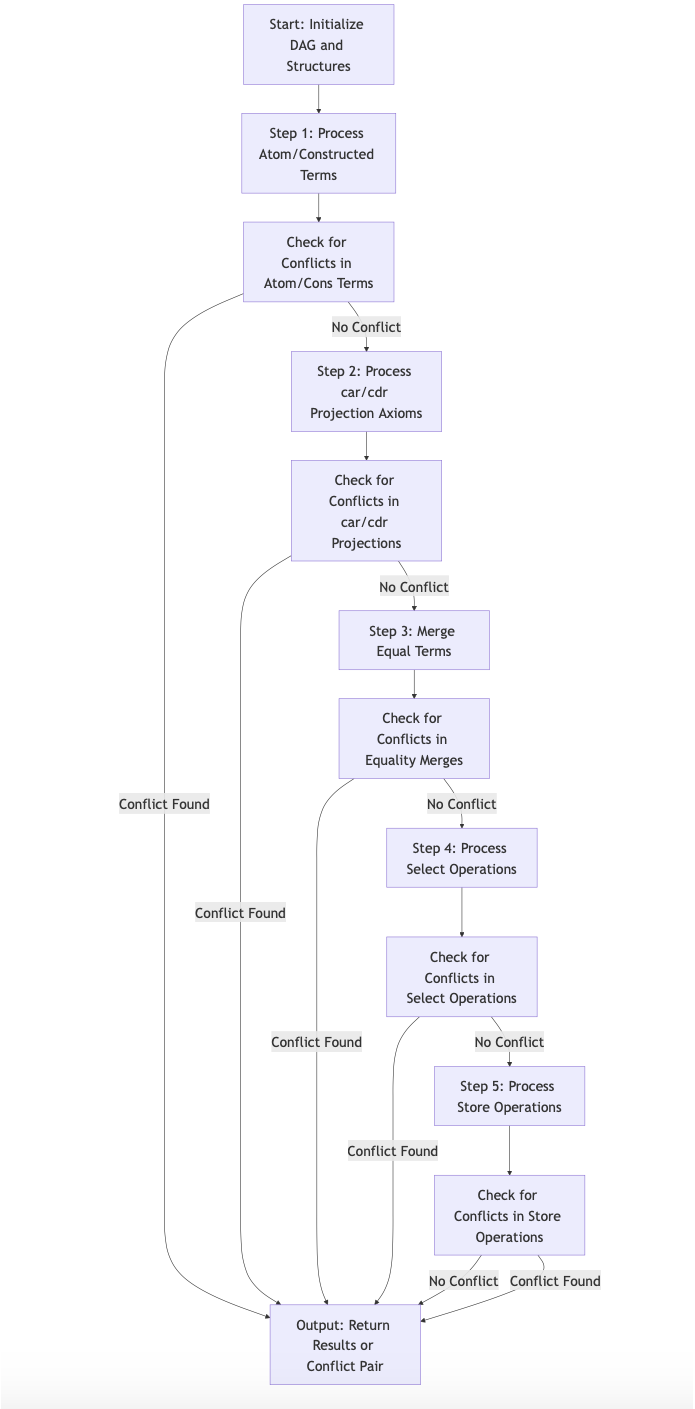

Figure 3: Nelson-Oppen & Congruence Closure DAG

1. **Step 1: Initialize the Algorithm and DAG**: The input formula is parsed, and a Directed Acyclic Graph (DAG) is constructed to represent its structure.
    - Nodes are created from the smallest to the largest element in the formula during the parsing process.
    - Each node contains the following components:
        - A *parent field* to track relationships between nodes.
        - A *find field* for resolving equalities.
        - Optional *arguments* to handle operations such as `car`, `cdr`, and `select`.
2. **Step 2: Atom and Cons Conflict Detection**: The algorithm examines all constructed lists for potential conflicts. For example, if `cons(x, y) = cons(a, b)` but `x != a`, a conflict is detected and reported.
3. **Step 3: List Theory Conflict Check (Car, Cdr)**: The algorithm checks the car, cdr operations on the created list cons. If inconsistencies or conflicts are found, they are reported as errors.
4. **Step 4: Equality and Inequality Processing**: The algorithm evaluates equalities (`=`) and inequalities (`!=`).
    - If two elements are equal, their nodes are merged.
    - If a conflict arises (e.g., `a = b` and `a != b`), the formula is marked as unsatisfiable.
5. **Step 5: Process Store Operations (Array Theory)**: The `store` operation is processed to ensure consistency in the array theory.
    - For example, `store(a, i1, v5)` guarantees that `select(a, i1)` returns `v5`.
    - If this condition is violated, a conflict is reported.
6. **Step 6: Process Select Operations**: The `select` operation is processed to ensure that the values returned are consistent with the array operations. If there is an inconsistency in select results, it is flagged as a conflict.
7. **Step 7: Final Satisfiability Check**: After processing all operations and transformations, the DAG is analyzed to determine whether the input formula is satisfiable. If no conflicts are detected, the formula is satisfiable. Otherwise, it is unsatisfiable.

Example:

```
F = car(x) = select(store(a, i, v), i); x != car(y)
```

DAG Representation:

- **Node 1:** `car(x)` – Represents the first `car` operation with the variable `x`.
- **Node 2:** `select(store(a, i, v), i)` – Represents selecting element `i` from the stored value `v` at index `i` of array `a`.
- **Node 3:** `car(y)` – Represents the second `car` operation with the variable `y`.

Process:

1. **Step 1: Parse the Formula and Create Nodes**
   - The formula is parsed, and each operation is converted into a node in the Directed Acyclic Graph (DAG).
   - `car(x)` forms a node, linked to the variable `x`.
   - `select(store(a, i, v), i)` is processed as a node with sub-nodes for both `store` and `select` operations.
   - `car(y)` forms another node, linked to `y`.
2. **Step 2: Check for Conflicts**
   - The formula includes the condition `x != car(y)`, which means `x` and `car(y)` cannot be equal.
   - If the algorithm detects `x = car(y)` anywhere in the DAG, a conflict is raised, and the formula is marked as unsatisfiable.
3. **Step 3: Process Array Operations**
   - The algorithm processes the `store` and `select` operations.
   - For `select(store(a, i, v), i)`, the `store` operation stores the value `v` at index `i` in array `a`.
   - The `select` operation should return the value `v` when queried with index `i`.
   - If `select(a, i)` doesn't return the stored value `v`, the algorithm identifies this as a conflict.
4. **Step 4: Evaluate the Formula**
   - After processing all operations, the algorithm evaluates the formula's satisfiability:
     - If no conflicts are found (i.e., there are no contradictions like `x = car(y)` and `x != car(y)`), the formula is satisfiable.
     - If any conflict occurs, the formula is marked as unsatisfiable.

4 How to Run the Project

The project integrates formula parsing, congruence closure calculation, and visualization. To execute the project, follow the steps below from the `ccDag` directory:

4.1 Prerequisites

- **Java Version**: Ensure Java 11.0.21 or later is installed.
- **Dependencies**: Include the required libraries: `gs-core-2.0.jar` and `gs-ui-swing-2.0.jar`.

4.2 Compile the Source Code

To compile the source code, execute the following command:

```
javac -cp libs/gs-core-2.0.jar:libs/gs-ui-swing-2.0.jar -d bin \
 src/App.java \
 src/GraphVisualization.java \
 src/CCAlgorithm/*.java \
 src/CCAlgorithm/parser/*.java \
 src/CCAlgorithm/bean/*.java \
 src/Transformation/Selection/*.java \
 src/Transformation/DNF2/*.java
```

4.3 Run the Application

After compiling, run the application using:

```
java -cp bin:libs/gs-core-2.0.jar:libs/gs-ui-swing-2.0.jar App
```

4.4 Run formula generator.

To generate a more complex formula with nested `select` and `store` operations:

```
java -cp src formulaGenerator 10 5 3 2 6 4 4 5 5
```

This will produce:

- 10 equalities, 5 disequalities, 3 atomic formulas, 2 negated atomic formulas, 6 predicate formulas, 4 negated predicate formulas, 4 `select` operations, 5 `store` operations, A maximum recursion depth of 5.

4.5 Directory Structure

The project structure is as follows:

- `src/`: Contains all source code files.
  - `CCAlgorithm/`: Implements the core congruence closure algorithm.
  - `CCAlgorithm/parser/`: Includes parsers for reading logical formulas.
  - `CCAlgorithm/bean/`: Defines data structures used in the algorithm.
  - `Transformation/Selection/`: Contains selection transformation logic.
  - `Transformation/DNF2/`: Includes transformations to Disjunctive Normal Form and Conjunctive Normal Form.
  - `GraphVisualization.java`: Visualizes the congruence closure DAG.
  - `App.java`: Entry point of the application.
- `bin/`: Stores compiled `.class` files.
- `libs/`: Contains external dependencies, such as GraphStream libraries.